

Problem-Based Learning Assignment

INDIVIDUAL ASSIGNMENT — All work must be your own. No group submissions.

Course	Object-Oriented Programming (C++)
Assignment Title	Airline Reservation & Flight Management System
Type	Problem-Based Learning (PBL) — Individual Work
Estimated Effort	20–25 hours over 2–3 weeks
Total Marks	100
Submission	Source code (.cpp / .h files), UML class diagram (PDF/PNG), and a 2–3 page report (PDF)

1. Learning Objectives

By the end of this assignment, you will be able to:

1. **Apply the four pillars of OOP** — encapsulation, inheritance, polymorphism, and abstraction — to model a real-world airline domain in C++.
2. **Design class hierarchies** using abstract base classes, virtual functions, and proper access specifiers (`public`, `protected`, `private`).
3. **Write memory-safe C++ code** using constructors, destructors, copy/move semantics, and the Rule of Three / Rule of Five where appropriate.
4. **Use operator overloading and friend functions** so custom types behave naturally in expressions and I/O.
5. **Apply exception handling** to deal with invalid input, sold-out flights, and runtime errors gracefully.
6. **Use the Standard Template Library (STL)** — containers (`vector`, `map`), iterators, and algorithms — to manage flights, passengers, and bookings.
7. **Persist data using file I/O** (text or binary) so the system remembers state between runs.
8. **Document and justify design decisions** with a UML class diagram and a written rationale.

2. Problem Scenario

A regional airline, **SkyLink Airways**, currently manages all bookings on spreadsheets. As the airline grows to new routes, this is becoming error-prone — overbooked flights, lost passenger records, and incorrect refunds are now common. The CTO has asked you, as a junior software engineer, to prototype a console-based **Airline Reservation & Flight Management System** in C++ that handles flights, passengers, ticket bookings, seat allocation, cancellations with refund rules, and end-of-day reports. The system must be reliable, extensible (so new flight or fare classes can be added later), and must remember its state between runs.

3. Problem Statement

You are required to design and implement a console-based C++ application that solves the scenario above by applying object-oriented principles. The system must demonstrate clear class design, sensible inheritance, and clean separation between data, logic, and user interaction.

4. Functional Requirements

4.1 Class Hierarchy (Mandatory)

- An abstract base class `Flight` with common attributes (flight number, origin, destination, departure date/time, total seats, available seats) and at least one pure virtual function (e.g., `calculateBaseFare()` and `displayDetails()`).
- At least three derived classes: `DomesticFlight`, `InternationalFlight`, and `CharterFlight`, each with their own pricing logic and specific attributes (e.g., visa requirements for international, contract holder for charter).
- A `Passenger` base class with derived classes `EconomyPassenger`, `BusinessPassenger`, and `FirstClassPassenger` — each with different baggage allowance, loyalty points multiplier, and cancellation refund percentage.
- A `Ticket` class linking a passenger to a flight, holding seat number, fare paid, and booking status.
- An `Airline` class that aggregates flights, passengers, and tickets and exposes the main operations.

4.2 Required Operations

1. **Flight management:** add, remove, search (by flight number, route, or date), and list all flights.
2. **Passenger management:** register, remove, and view a passenger's booking history.
3. **Booking:** assign an available seat; reject if the flight is full or the passenger already has a ticket on that flight.
4. **Cancellation & refund:** compute refund polymorphically — different rules per passenger class and how close to

departure the cancellation occurs.

5. **Reports:** "today's departures", "occupancy percentage per flight", and "top 5 highest-revenue flights this month".
6. **Persistence:** on exit, save the full state to a file; on startup, restore it.
7. **Menu-driven interface:** a clean console menu with input validation.

4.3 Required OOP Features

- At least one **abstract class** with pure virtual functions.
- **Polymorphism** via virtual functions (e.g., fare calculation, refund calculation, display).
- **Operator overloading** — overload `<<` for displaying any `Flight` or `Ticket`, and `==` for comparing tickets by ID.
- **Function or class templates** — at least one (e.g., a generic search utility that works on flights or passengers).
- **Exception handling** — define and throw at least two custom exception classes (e.g., `FlightFullException`, `InvalidCancellationException`).
- **STL usage** — use `std::vector`, `std::map` (or `unordered_map`), and at least one STL algorithm (e.g., `sort`, `find_if`).
- Proper **destructors** and resource management — no memory leaks.

5. Non-Functional Requirements

- Code must compile with `g++ -std=c++17 -Wall -Wextra` with no warnings.
- Code must be split across multiple files (header + implementation) with proper include guards.
- A `Makefile` or build instructions must be provided.
- Meaningful identifiers, consistent indentation, and inline comments where logic is non-obvious.

6. Deliverables

1. **Source code** — all `.h` and `.cpp` files, organized in folders.
2. **UML class diagram** — showing classes, attributes, methods, and relationships (inheritance, aggregation).
3. **Design report (2-3 pages)** covering: design choices, why each OOP feature was used where, known limitations, and a list of test cases you ran with their outcomes.
4. **Sample data file** with at least 10 flights and 8 passengers for demonstration.
5. **README.md** explaining how to build and run the program.

7. Submission Guidelines

- Submit a single `.zip` archive named `OOP_PBL_<StudentID>.zip` on the LMS.
- Late submissions: **-10% per day**, up to a maximum of 3 days; not accepted thereafter.
- **Individual viva is mandatory.** Each student will demonstrate the running program in a 10-minute viva and answer questions on any line of the code. Marks are awarded based on the viva — a working program you cannot explain will receive zero.
- This is strictly individual work. Sharing code with classmates — in either direction — is treated as plagiarism. You may use AI tools to clarify concepts or debug, but every line you submit must be one you wrote and can defend. Submissions are checked with similarity tools.

8. Suggested Timeline

- **Week 1:** Requirements analysis, UML diagram, base classes, and a skeleton project that compiles.
- **Week 2:** Implement core operations, polymorphism, exception handling, and STL integration.
- **Week 3:** File persistence, reports, testing, polish, report writing, and viva preparation.

9. Assessment Rubric (Total: 100 Marks)

Criterion	Marks	Excellent (90-100%)	Good (70-89%)	Satisfactory (50-69%)	Needs Work (0-49%)
Class Design & UML	15	Clean hierarchy with appropriate abstraction; UML accurately reflects code; relationships clearly modeled.	Mostly sound design with minor abstraction issues; UML matches code with small discrepancies.	Hierarchy works but is shallow or has misplaced responsibilities; UML partially reflects code.	Poor or missing hierarchy; UML missing or incorrect.
Encapsulation & Abstraction	10	All data members private; access via well-chosen getters/setters; abstract base class used meaningfully.	Mostly private members; abstract class present and used.	Some members public; abstraction present but underused.	Public data members, no abstract class, or abstraction not applied.
Inheritance & Polymorphism	15	Multiple meaningful derived classes; virtual functions used correctly; runtime polymorphism clearly demonstrated.	Inheritance applied; polymorphism works in most places.	Basic inheritance; polymorphism limited or partly incorrect.	No real inheritance; virtual missing or misused.
Operator Overloading & Templates	10	Both required overloads work cleanly; template is genuinely generic and useful.	Overloads and template present and functional.	Implemented but with minor bugs or limited usefulness.	Missing or broken.
Exception Handling	10	Custom exceptions used; all error paths handled gracefully; no crashes on bad input.	Custom exceptions used; most error paths handled.	Some exception handling; crashes on edge cases.	No exception handling or program crashes frequently.
STL Usage	5	Multiple containers and at least one algorithm used idiomatically.	Required containers used correctly.	Containers used but in non-idiomatic ways.	STL not used or misused.
File Persistence	10	Robust save/load; handles missing/corrupt files; round-trip preserves all state.	Save/load works for normal cases.	Partial persistence or loses some data.	No persistence or non-functional.
Functionality & Correctness	10	All required operations work; edge cases handled; reports correct.	All major operations work; minor bugs.	Core operations work; several bugs.	Major functionality missing or broken.
Code Quality & Memory Safety	5	No leaks; clean style; consistent naming; multi-file structure with header guards.	Mostly clean; minor style issues; no major leaks.	Works but messy; possible small leaks.	Memory leaks; single-file dump; poor style.
Documentation & Report	5	Clear report; design choices justified; README complete; helpful comments.	Adequate report and README.	Minimal documentation.	Missing or incoherent documentation.
Viva & Demonstration	5	Confidently explains every part; answers follow-up questions on design trade-offs.	Explains most parts well.	Explains basics; struggles with deeper questions.	Cannot explain own code.

10. Academic Integrity Statement

This assignment is an opportunity to develop your own engineering judgment. You may discuss high-level concepts with peers, but writing code together, sharing files, or copying any portion of another student's work is strictly prohibited. AI tools may be used to learn concepts or debug your own code, but all submitted code must be code you understand and

can defend in the viva. If you used AI assistance, briefly note where in your report. Suspected violations will be investigated and may result in zero marks for the course component.

11. Helpful Tips

- Start with the UML diagram before writing any code — it will save you hours later.
- Get a minimal end-to-end version working (one flight type, one passenger class, no persistence), then grow from there.
- Use `std::unique_ptr` or `std::shared_ptr` instead of raw `new/delete` wherever possible.
- Write small test cases as you go — don't leave testing for the last day.
- Commit your work to a private git repository so you don't lose progress.

Good luck! The goal of this PBL is not just a working program — it is the engineering thinking behind it. Make decisions deliberately, and be ready to defend them on your own.